2. Strong Migrations checks for table modifications *and* data backfills that would extend the duration of an exclusive lock
3. When removing or renaming a column or table, Strong Migrations checks whether this will create a problem with the Schema Cache
4. Strong Migrations prevents changing the type of a column to an incompatible type.
5. When Adding or Dropping an Index without the `CONCURRENTLY` keyword, Strong Migrations prompts the developer to use CONCURRENTLY.

How does it work?

`Strong Migrations` hooks into the Rails Migration process. Once it's added to a Rails project, every time a developer runs `bin/rails db:migrate` it checks any pending modifications for safety.

A Ruby Exception is raised when a potentially dangerous migration is detected which prevents the developer from applying it.

This forces the developer to either abandon it, rewrite it, or explicitly opt out of the safety check. To opt out, the developer can wrap their changes in a `safety_assured` Ruby block. Review the `db/migrate` directory of Rideshare to see some places with opt out examples.

Most of the time, `Strong Migrations` helpfully catches unsafe migrations.

Because this whole cycle may be conducted before ever submitting a change for review, it creates an opportunity for developers to learn privately about this information. This tight feedback loop can contribute to scaling knowledge on the team around safe and unsafe database changes.

Next you'll look at a specific example from Rideshare.

Trips can have a rating of integer values between 1 and 5. As an Active Record Validation, Rideshare uses a "numericality" validator for the Trip model that makes sure the value is between 1 and 5. The rating value can also be nil since trips aren't rated until after the Driver completes the trip.

Review the `rating` field validation in `app/models/trip.rb`.

The rating validation is useful but it's implemented on the client side Rails application.

The rating validation can also be implemented in PostgreSQL using a `CHECK` constraint.[3] The CHECK constraint used here has already been added to

3. https://www.postgresql.org/docs/current/ddl-constraints.html#DDL-CONSTRAINTS-CHECK-CONSTRAINTS